Звіт з програмного проєкту 4. Варіант 7

Виконавець: Гламазда Єлизавета, група К-27.

Варіант 7, m = 3 поля. Частки операцій від усіх операцій: поле 0 — read 5%, write 5%; поле 1 — read 30%, write 5%; поле 2 — read 24%, write 1%; запит string — 30%. Потрібно розробити потокобезпечну структуру з 3 цілих полів без атомарних змінних, оптимізовану під ці частоти, і виміряти час для трьох файлів дій при 1, 2 і 3 потоках: а) частоти за умовою; б) усі частоти рівні; в) підібрані вручну (read 0 — 40%, write 0 — 50%, решта операцій по 1%, string — 6%). Усього 6 000 000 операцій незалежно від кількості потоків, кожен потік має власні дані.

Три std::shared_mutex, по одному на кожне поле. Read бере shared_lock, write — unique_lock лише свого поля, operator string — shared_lock на всі три поля в порядку 0, 1, 2 (значення копіюються, м'ютекси відпускаються, і лише потім формується рядок). Обґрунтування: записів лише 11% від усіх операцій, а читання та string (разом 89%) сумісні між собою, тому блокують лише записи. Один спільний м'ютекс серіалізував би 100% операцій, а звичайні м'ютекси на поля змушували б string (30%) блокувати всі поля й читачів. Фіксований порядок блокування та те, що писальник тримає лише один м'ютекс, виключають взаємоблокування. Слоти полів вирівняно на 64 байти (alignas(64)), щоб м'ютекси різних полів не ділили рядок кешу.

Середній час виконання, мс (Release x64, 5 запусків):

Файл	1 потік	2 потоки	3 потоки
а (за умовою)	487	490	498
б (рівні частоти)	295	313	334
в (вручну)	207	201	193

Висновки. Таблиця не показує очікуваного прискорення від додаткових потоків: для файлів а і б час трохи зростає, для в зменшується лише приблизно на 7%. Імовірна причина — дуже короткі критичні секції (десятки наносекунд): вартість захоплення й відпускання м'ютексів та конкуренція потоків за стан одного й того самого м'ютекса (навіть shared_lock його змінює) перекривають виграш від паралелізму. Співвідношення між файлами очікуване: а найповільніший, бо string (30%) — найдорожча операція (три блокування та створення рядка), у б таких операцій 14%, у в лише 6%. Схема коректна (без data race і взаємоблокувань) і не серіалізує читачів, але на настільки коротких операціях масштабується слабко.